Mitsubishi Electric Sales Intelligence System

System overview and operating guide

Changes for the Better

A practical guide to forecasting, demand classification, replenishment planning, data lineage, and teammate setup.

Purpose and scope

Missubibi is a Mitsubishi Electric Thailand sales intelligence and demand-planning platform. It connects verified Mitsubishi sales history in MySQL to forecasting, item-level demand classification, and lead-time inventory planning. This document explains what the software does, what controls are available, how the data moves through the system, and how teammates can start the current version with the matching database snapshot.

The central planning question is simple: what has sold, what is likely to sell next, which items need attention, and when should replenishment be ordered? The system keeps the evidence and the planning assumptions visible together so a forecast can be explained rather than treated as a black box.

Contents

- 1 System at a glance
- 2 Forecast workspace
- 3 Model performance
- 4 Demand classifier
- 5 Lead-time inventory
- 6 Data and coverage
- 7 CSV imports and data quality
- 8 Architecture and operations
- 9 Teammate setup and database synchronization
- 10 Interpretation rules and operational boundaries

1 System at a glance

The dashboard is organized as one planning loop. A user can begin with the data check, move into a forecast for a particular slice of the business, inspect the evidence behind that forecast, prioritize items with the classifiers, and finish with a replenishment queue.

Section	What it answers	Main user action
Forecast workspace	What demand or revenue is likely for a selected business slice?	Set scope, measure, frequency, calendar, horizon, and model; then run the forecast.
Model performance	Which forecasting engine has the strongest historical evidence?	Compare accuracy, WAPE, MASE, folds, weights, status, and forecast shape.
Demand classifier	Which items are likely to be high demand or sold in the next eligible month?	Train or refresh the September models, check items, and review borderline probabilities.
Lead-time inventory	When should stock be ordered and how much is suggested?	Set lead time and coverage assumptions, edit on-hand stock, and review the queue.
Data and coverage	What data is connected and how trustworthy is the current planning view?	Inspect dimensions, date coverage, source rows, duplicates, missing prices, and fingerprints.

The backend uses one canonical SQL adapter for forecasts, classifiers, inventory planning, and data coverage. That shared adapter is what lets a user trace a result back to the same source snapshot and filtering rules.

2 Forecast workspace

The Forecast workspace is the main analysis screen. It lets a planner move from a company-wide view to a specific item, customer, product category, industry, or vendor view, then choose the time and value representation that matches the business question.

Available controls

Control	Options
Scope	Company, item / SKU, product category, customer, industry, or vendor.
Combined filters	Item, customer, industry, and vendor filters can be applied together.
Measure	Revenue in THB or sales volume in units.
Frequency	Daily, weekly, monthly, quarterly, or yearly.
Calendar	Calendar year or fiscal year beginning in April.
Horizon	1 to 120 periods; yearly and quarterly horizons have tighter limits.
Model	Trend, SARIMA, Random Forest, LSTM, XGBoost, or Ensemble.

How to use the forecast

1. Choose the scope and select the item, customer, product, industry, or vendor value when the scope is not the full company.
2. Choose revenue or units, then select the daily, weekly, monthly, quarterly, or yearly view.
3. Choose the calendar convention and set the horizon with the slider.
4. Select a model and click Run forecast. The request runs as a background job and the page polls for the completed result.
5. Read actual history, model forecast, prediction range, accuracy, and totals together.
6. Drag forecast points or edit the period table when you want to test a higher or lower planning scenario.

Scenario meaning: a scenario is an editable planning version of the model forecast. It changes the planning values and scenario fit indicator, but it never rewrites the historical validation score.

Forecast output can be exported to CSV with the model forecast, planning scenario, lower prediction range, and upper prediction range for each future period.

3 Model performance

The Model performance section compares the six forecast engines using chronological held-out validation against the repaired Mitsubishi SQL snapshot. The displayed scores are calculated evidence, not default ratings and not values adjusted by the planning scenario.

Model	Role	Held-out accuracy
Trend	Interpretable baseline	78.72%
SARIMA	Seasonal statistical model	83.48%
Random Forest	Nonlinear demand patterns	84.42%
LSTM	Sequential neural network	86.41%
XGBoost	Gradient-boosted trees	75.87%
Ensemble	Validation-weighted blend	85.20%

What the comparison table shows

- Accuracy: max(0, 100 minus validation WAPE).
- WAPE: total absolute forecast error divided by total absolute actual demand.
- CV MASE: error scaled against a seasonal-naive reference.
- Folds: the number of expanding chronological validation windows.
- Weight: the contribution used when an ensemble forecast is built.
- Outlook: a sparkline showing the model forecast shape for the selected series.

Two independent validation runs on the same 265,135-row canonical SQL view reproduced the forecast scores to eight decimal places. A score should change when the SQL data, model code, model configuration, output granularity, or validation protocol changes. A page refresh or scenario edit should not change it.

Prediction ranges are useful planning ranges. Weighted ensemble bounds are approximate and are not claimed to be calibrated statistical 90 percent intervals.

4 Demand classifier

The classifier section sits beside the numeric forecast. It helps prioritize item review by answering two different questions: which items are likely to be high demand, and which items are likely to sell or not sell in the next eligible month.

Model	Target	Held-out accuracy	Other diagnostics
Neural network rev9	High demand	96.92%	Balanced accuracy 83.73%; baseline 95.81%; AUC about 0.972; up to 5,706 items.
Tabular transformer rev8	Sold / not sold	89.61%	Balanced accuracy 81.80%; baseline 86.49%; AUC about 0.915; 5,620 items with sufficient history.

How to use the classifier

- Choose Quick training for up to 12 epochs or Full training for up to 100 epochs.
- Search for one item / SKU or leave the field blank to score all eligible items.
- Click Check demand to request probability rows from both classifiers.
- Use the borderline review queue to focus on probabilities that need human review.
- Read held-out accuracy, balanced accuracy, ROC AUC, the last-month baseline, held-out item-month count, and the confusion matrix.

Classifier outputs prioritize attention; they do not replace the revenue or unit forecast. If a model export is missing or stale, the system returns an unavailable state instead of fabricating probabilities.

5 Lead-time inventory

Lead-time inventory connects the sales run rate to the date a purchase order needs to be placed and the date stock is expected to arrive ready for use. Demand comes from the canonical Mitsubishi SQL history. Supplier lead time and warehouse on-hand are explicit planner inputs.

Planning input	Control range or source	Effect
Supplier lead time	1 to 365 days	Determines demand during the supply wait and order-by date.
Safety stock	0 to 12 months	Adds a reserve above lead-time demand.
Forward coverage	0.25 to 24 months	Sets target demand coverage after arrival.
On-hand units	Editable per item	Reduces the suggested order and determines Covered or Order now.

Planning calculations

```
Demand run rate = 70% trailing three-month demand + 30% trailing twelve-month demand
Lead-time demand = demand run rate × supplier lead-time days
Reorder point = lead-time demand + safety stock
Target stock = lead-time demand + safety stock + forward coverage
Suggested order = target stock − on-hand units
```

The replenishment queue shows item, monthly run rate, demand in lead time, safety stock, reorder point, editable on-hand units, suggested order quantity, order-by date, expected arrival date, and status.

The source Mitsubishi files do not contain purchase orders, receipts, supplier lead times, or warehouse balances. The interface keeps those values visible as planning inputs instead of hiding assumptions inside the calculation.

6 Data and coverage

The data section is the provenance check before a forecast is interpreted. It shows what is connected to MySQL, which dimensions are available, and which source-quality conditions remain visible in the planning view.

Source or view	Verified content
Customer.xlsx	2,159 customer rows.
Item.xlsx	5,706 item rows.
Sale Order History.xlsx	272,000 sales-order-history lines.
RawData_Combined.xlsx	Exact copies of the three original sheets.
Sales_Order_Merged in MySQL	Original sales fields and repaired dates, prices, quantities, derived revenue, and quality flags.
Canonical forecast view	265,135 rows after removing extra exact duplicate copies for modeling.

Coverage and quality signals

- Source coverage runs from April 2021 through March 2025 across 48 monthly periods.
- The source contains 87 product categories and 69 industries.
- The database preserves the original sales dates from 2021-04-01 through 2025-03-31.
- 6,865 extra exact duplicate lines are flagged; 11,928 rows belong to duplicate groups.
- 13 items have missing list prices; 15 sales lines retain null list-price-derived values.
- The original Mitsubishi source has no vendor relationship, so vendor-level views require an imported mapping.

The first SQL load had stored zero dates and carriage returns in month labels. Those artifacts were repaired from the original source workbooks before the current seed was generated.

7 CSV imports and data quality

The Import CSV button is available in the top-right controls. Importing is preview-first: the file is read, normalized, validated, and shown as sample rows before anything is committed.

Import type	Required columns	Optional columns or behavior
Sales	month, item, actual_units, revenue	product, customer, industry, vendor; common source aliases are accepted.
Vendor mapping	item, vendor	Each item must map to one vendor.
Inventory mapping	item plus on_hand_units or lead_time_days	Either numeric inventory column may be supplied.

Import safeguards

- UTF-8 CSV validation with a 20 MB file limit and a 250,000-row limit.
- Date, quantity, revenue, item, vendor, on-hand, and lead-time validation.
- Append or replace-active modes for sales overlays.
- Exact duplicate detection and idempotent import tokens.
- Original SQL source tables are never overwritten.
- Imported overlays refresh the forecast and classifier data adapter after commit.

Sales CSV template header:

```
month,item,product,customer,industry,vendor,actual_units,revenue
```

8 Architecture and operations

Layer	Implementation
Frontend	React, TypeScript, and Vite on port 5174.
Backend	FastAPI, SQLAlchemy, and PyMySQL on port 8100.
Database	MySQL database named missubibi.
Forecast execution	Bounded background jobs with isolated model workers.
Persistence	Local job records, forecast registry, fingerprints, diagnostics, and results.
Scheduler	Hourly check for changed data or a new UTC month; refreshes requested configurations.
Protection	Optional PLANNING_API_KEY for API routes.

Model workers isolate native statistical, machine-learning, and neural-network libraries so one engine failure or native threading conflict does not take down the API process. The scheduler refreshes previously requested configurations; it does not launch exhaustive training for every possible item, customer, industry, and vendor combination.

The model registry versions reproducible forecast runs and active revisions. It stores parameters, data and code fingerprints, diagnostics, and results rather than a complete binary copy of every fitted estimator.

9 Teammate setup and database synchronization

The shared GitHub branch contains the repaired Mitsubishi SQL seed, the dashboard implementation, the startup scripts, and the database freshness check. Teammates should use the working branch rather than the older default branch.

```
git clone -b codex/forecast-planning-workspace
https://github.com/sokolovxf/missubibi.git
cd missubibi
./start-local.sh --sync-db
```

For an existing clone:

```
git fetch origin
git checkout codex/forecast-planning-workspace
git pull
./start-local.sh --sync-db
```

The startup command compares the checked-out schema.sql and seed.sql fingerprints with a marker recorded inside MySQL. If the local database is already current, it is left untouched. If it is missing or stale, the script rebuilds the local missubibi database from the checked-out files and then starts the backend and frontend.

Windows PowerShell:

```
.\start-local.ps1 -SyncDatabase
```

The database refresh is intentionally explicit because it drops and recreates the local missubibi database. The metadata marker is excluded from future database exports, so it does not become part of the shared source seed.

Service	Address
Frontend	http://localhost:5174
Backend	http://127.0.0.1:8100
Health check	http://127.0.0.1:8100/api/health

10 Interpretation rules and operational boundaries

These rules keep the dashboard results honest and prevent a planning control from being mistaken for a model guarantee.

- Forecast accuracy is historical held-out evidence, not a guarantee of future accuracy.
- Scenario fit is separate from model validation accuracy.
- Daily and weekly forecast values may be allocated from the monthly modeling contract, while native transaction-date history is available as a separate view.
- Vendor-level planning requires a vendor mapping because the original Mitsubishi source does not include vendor relationships.
- Inventory recommendations depend on planner-supplied on-hand units and supplier lead-time values.
- Classifier signals can be exposed experimentally, but they are disabled by default in the main forecast service.
- Weighted ensemble bounds are approximate planning ranges rather than calibrated statistical intervals.
- The current application is a local single-process deployment with local MySQL and local job storage. Optional API-key protection is available through PLANNING_API_KEY.

Quick reference

Need	Use
Forecast a portfolio or slice	Forecast workspace → scope → measure → frequency → horizon → model → Run forecast.
Compare model evidence	Model performance → inspect accuracy, WAPE, MASE, folds, weights, and sparklines.
Prioritize item review	Demand classifier → train or refresh → select item or all → Check demand → review borderline rows.
Plan a replenishment	Lead-time inventory → set assumptions → apply → edit on-hand → review suggested order and dates.
Verify source coverage	Data & coverage → inspect dimensions, date coverage, duplicate flags, missing prices, and fingerprint.
Load new operational data	Import CSV → preview → validate → append or replace-active → commit.
Start with current SQL data	Run ./start-local.sh --sync-db or .\start-local.ps1 -SyncDatabase.

The complete implementation is available on the GitHub branch codex/forecast-planning-workspace.